

Data-Driven Forward Kinematics Modelling of Concentric Tube Continuum Robot



Data Mining and Machine Learning Project Report

Ali Syaugi Bilfagih (701967) – a.bilfagih@studenti.unipi.it

Master's Degree in Bionics Engineering
Academic Year 2024/2025

Table of Contents

1.	Introduction	1
2.	Dataset Overview	1
3.	Data Preprocessing	2
3.1.	Data Cleaning	2
3.2.	Exploratory Data Analysis.....	2
3.3.	Splitting Dataset	4
4.	Models	4
4.1.	Linear Regression	4
4.2.	Random Forest Regressor	5
4.3.	Extra Trees Regressor.....	7
5.	Discussion	8
6.	Summary.....	8
	References.....	9

1. Introduction

Continuum Robots are type of robot which are characterized by its absence of rigid-links and joint and continuous appearance. Among types of Continuum Robots, Concentric Tube refers to the mechanism which utilizes curved multi-tube catheter structures. There are difficulties to model the kinematics of Concentric Tube Continuum Robot (CTCR) using traditional physical based approach due to its non-linearity behaviour of elastic interactions between the flexible tubes. Data-driven approach can be utilized to be an alternative method, by training machine learning model with the robot's dataset on joint configurations and space representations, thus smaller error motion could be achieved.

The Continuum Robotics Laboratory from University of Toronto, Canada, established a CTCR public dataset with 100000 joint configurations. The benchmark of the dataset was conducted by utilizing Feedforward Neural Network (FFN) to model forward kinematics of Concentric Tube Continuum Robot. As results, 0.7 mm error position and 6.49° orientation error was achieved [1].

In this project, with the same dataset, several regression techniques including Linear Regression, Random Forest Regressor, and Extra Trees Regressor are utilized to model predict the tip pose of the Concentric Tube Continuum Robot. In general, the process includes data preprocessing, cross-validation and hyperparameter tuning for model training and selection, then comparison among models used to determine which one has the best performance.

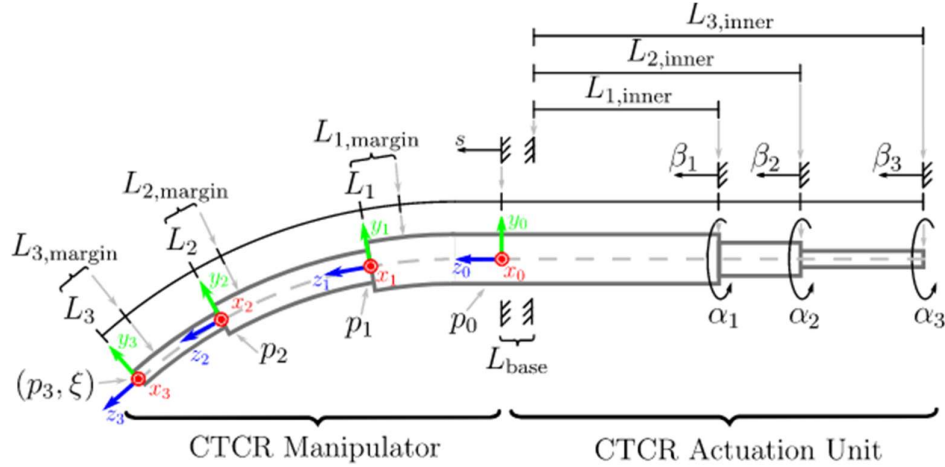


Figure 1.1 The CTCR Notation

2. Dataset Overview

The dataset is a CSV file with size of 56.5 MB, which consists of 100000 row and 40 data points including six absolute joint values, six relative joint values, pose of the base, pose of the proximal sensor attached to the outermost tube, pose of the sensor attached to the middle tube, pose of the distal sensor attached to the most inner tube. The annotation is provided on Table 2.1, whereas the visualization of the joint configurations and shape representations are available in Figure 1.1.

Table 2.1 Annotation of the dataset in the CSV file

Joint configuration $\mathbf{q}_k$ α_i in rad β_i in mm												i^{th} pose for $i = 0, 1, 2, 3$ (These columns repeat four times)							
												Joint configuration difference $\Delta \mathbf{q}_k = \mathbf{q}_k - \mathbf{q}_{k-1}$ $\Delta \alpha_i$ in rad and $\Delta \beta$ in mm				Position $\mathbf{t}$ $[x, y, z]^{\top}$ in mm			
α_1	β_1	α_2	β_2	α_3	β_3	$\Delta \alpha_1$	$\Delta \beta_1$	$\Delta \alpha_2$	$\Delta \beta_2$	$\Delta \alpha_3$	$\Delta \beta_3$	x_i	y_i	z_i	η_i	$\varepsilon_{i,1}$	$\varepsilon_{i,2}$	$\varepsilon_{i,3}$	

3. Data Preprocessing

Several data preprocessing steps are performed, including data cleaning to ensure there was no missing data, exploratory data analysis to understand general patterns of the data, and splitting dataset into train set and test set before finally will be used by the regression models. Another data preprocessing step such as normalization was not performed to keep the values as there are, therefore smaller error rate can be achieved.

3.1. Data Cleaning

To observe whether there is a missing value in the dataset or not, data cleaning was performed. Since the dataset was taken in a controlled laboratory environment, there is no missing value detected in the dataset.

3.2. Exploratory Data Analysis

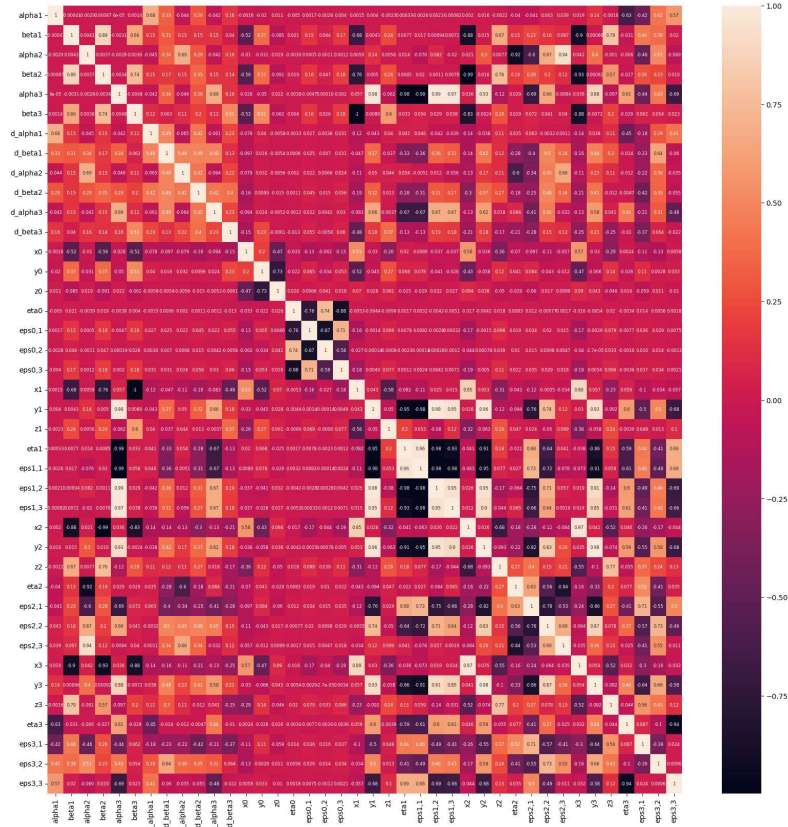


Figure 2.1 Heatmap of dataset correlations

To understand feature distributions, correlation between parameters, and inspect if there is any outlier in the dataset, Exploratory Data Analysis was performed. First, general heatmap of the dataset was generated as in Figure 2.1. Then, the heatmap is simplified only among joint configurations and tip pose $[x_3, y_3, z_3, \eta, \varepsilon_1, \varepsilon_2, \varepsilon_3]^T$ as shown in Figure 2.2.

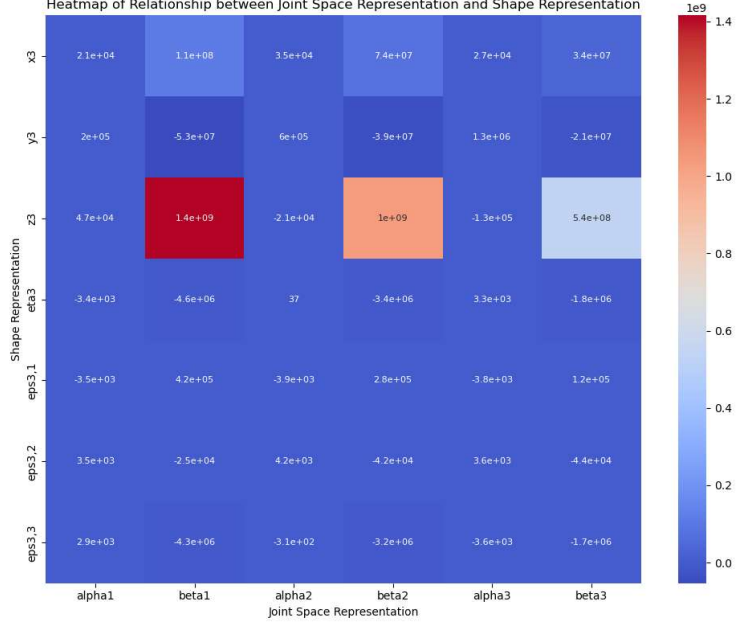


Figure 2.2 Simplified Dataset Heatmap

The outlier on the dataset was observed by using boxplot, as shown in Figure 2.3. These outliers for each position points and orientation angles indicate a phenomenon which is called as ‘snapping’. This phenomenon occurred likely due to acquisition data that took days and shortened path without considering the required energy.

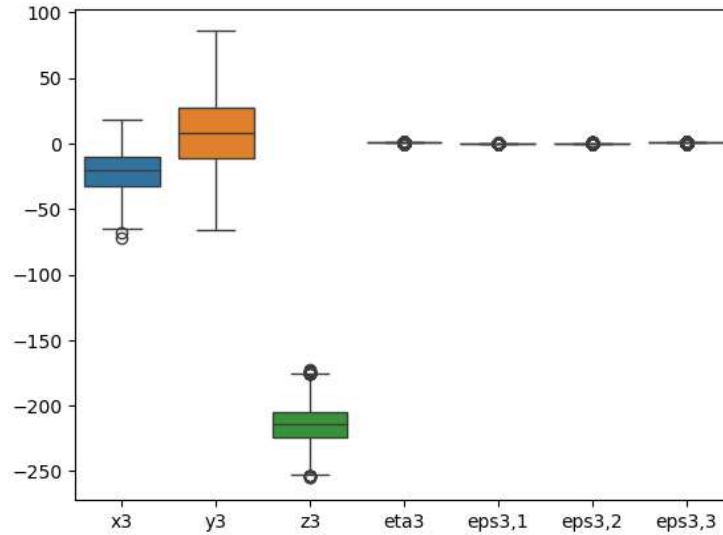


Figure 2.3 Boxplot capturing outlier on the dataset

3.3. Splitting Dataset

The dataset is separated into position and orientation set, since there are different error measurement metrics for each. To avoid data leakage, the dataset is thus split into 2 parts, training set and test set, with proportion 80:20 respectively. This will allow generalized evaluation and prevent overfitting of the models.

4. Models

The project employs traditional machine learning techniques, starting with Linear Regression, then followed with two decision tree based regressors, including Random Forest Regressor and Extra Trees Regressor. For each, Grid Search hyperparameter tuning and k-fold cross-validation using k=10 are implemented. Cross-validation is implemented to asses how well the model generalizes to independent dataset, whereas hyperparameter tuning decides which parameters results the best model performance.

Other than R^2 score and Mean Squared Error (MSE), the metric used to evaluate the performance of each model are position and orientation errors. For position, it is shown as equation below:

$$e_{t,i} = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (z_i - \hat{z}_i)^2}$$

where it matches the equation of Mean Absolute Error (MAE), thus, they can be interchangeably used. While for orientation, there is another equation as notated below:

$$e_\theta = \min\{2 \arccos(\eta\hat{\eta} + \varepsilon_1\hat{\varepsilon}_1 + \varepsilon_2\hat{\varepsilon}_2 + \varepsilon_3\hat{\varepsilon}_3), 2 \arccos(\eta\hat{\eta} - \varepsilon_1\hat{\varepsilon}_1 - \varepsilon_2\hat{\varepsilon}_2 - \varepsilon_3\hat{\varepsilon}_3)\}$$

where it respects the antipodal property of quaternions.

4.1. Linear Regression

Linear regression is statistical model that predicts a continuous value based on one or more independent attributes. Generally, linear regression can be modelled as equation below:

$$\hat{y} = w^T x_i + b$$

Linear regression assumes that the output value y is a linear weighted summation of the input attribute x_i values, offset by the bias scalar b .

In practical applications, there are several parameters can be tuned to obtain a linear regression model with the best performance. The parameters are including:

`fit_intercept` : Determines whether the model should calculate the intercept (bias term). If it is True, the model will learn an intercept term, which allows it to fit data that is not centered around the origin. Otherwise, the model assumes the data is already centered (mean of zero), and no intercept is added.

- n_jobs : Specifies the number of CPU cores to use for computation. The 'None' value will make the model use 1 core (default behavior). When a specific number was assigned, that's the total number used. This is useful for speeding up training, especially when there are multiple target variables.
- positive : This will force the model to learn only positive coefficients. If it was True, all learned weights will be ≥ 0 . Otherwise, coefficients can be positive or negative.

GridSearchCV was executed to implement both k-folds and hyperparameter tuning, with range:

- fit_intercept : True, False
n_jobs : 1, 5, 10, 15, None
positive : True, False

After running it, the best parameters was found with True fit_intercept, n_jobs equals to 1, and positive equals to False. R2 score obtained was 0.92. Then, the model with specific parameters was used to predict both the position and the orientation. Table 4.1 shows the evaluation values of the model.

Table 4.1 Evaluation Metric of Linear Regression Model

Position	
R2 Score	0.9199
Mean Squared Error	21.691
Error Position	3.451 mm
Orientation	
R2 Score	0.832
Mean Squared Error	0.0019
Error Orientation	14.17°

4.2. Random Forest Regressor

Random Forest (RF) is a popular ensemble learning method developed to address the limitations of traditional Decision Tree algorithms. It works by training multiple decision trees simultaneously, which helps reduce both bias and variance in the model. The RF algorithm constructs the forest by randomly drawing N bootstrap samples from the original dataset. For each sample, an unpruned regression tree is built. Instead of evaluating all available predictors at each split, only K randomly selected predictors are considered. This process is repeated until C trees are generated. Predictions for new data are then made by averaging the outputs of these C trees. By using bagging—training trees on different subsets of the data—RF enhances tree diversity and lowers the overall variance of the model [2].

$$\hat{y} = \frac{1}{C} \sum_{i=1}^C T_i(x)$$

The mathematical notation of Random Forest Regressor is notated by the equation above, where $\hat{y}$ is the predicted value, C is the total number of trees in the forest, $T_i(x)$ is the prediction made by the $(i) - th$ decision tree for the input x .

Several parameters can be adjusted to optimize the performance of a linear regression model. These parameters include:

- `n_estimators` : It specifies the number of decision trees in the forest. More trees generally improve performance and stability, but also increase computation time. After a certain point, adding more trees yields diminishing returns.
- `max_depth` : It sets the maximum depth each tree can grow to. If set to None, trees will grow until all leaves are pure (i.e., contain only one class or value) or until they contain fewer samples than `min_samples_split`. Shallower trees (smaller `max_depth`) reduce overfitting but may underfit, while deeper trees can capture more complex patterns but risk overfitting.
- `max_features` : It controls how many features are considered when looking for the best split at each node. Lower values increase randomness and diversity among trees, which can improve generalization, whereas higher values reduce randomness, making trees more similar and potentially increasing overfitting.

GridSearchCV was executed to implement both k-folds and hyperparameter tuning, with range:

- `n_estimators` : 10, 50, 100, 500
- `max_depth` : 5, 10, 20, None
- `max_features` : None, "log2", "sqrt"

The best parameters was found with `n_estimators` equal to 500, `max_depth` equals to None, and `max_features` equals to log2. R2 score obtained was 0.991. The process is repeated independently for position and orientation model training, to ensure if there would be any difference for each, where eventually they have the same parameters. Table 4.2 shows the evaluation values of the model.

Table 4.2 *Evaluation Metric of Random Forest Regressor Model*

Position	
R2 Score	0.991
Mean Squared Error	3.00
Error Position	1.17 mm
Orientation	
R2 Score	0.965
Mean Squared Error	0.00044
Error Orientation	8.968°

4.3. Extra Trees Regressor

Extremely Randomized Trees (Extra Trees) is a relatively recent machine learning technique that builds upon the widely-used Random Forest algorithm. It is specifically designed to reduce the risk of overfitting. Like Random Forest, Extra Trees trains multiple decision trees using random subsets of features. However, unlike Random Forest, it introduces additional randomness by selecting split thresholds at random rather than searching for the optimal split value for each feature [3].

GridSearchCV was executed to implement both k-folds and hyperparameter tuning, with range:

```

n_estimators : 10, 50, 100, 300
max_depth    : 5, 10, 20, None
max_features : None, log2, sqrt

```

The number of trees is decreased to prevent very high computational complexity, and observe whether fewer number of trees is enough for Extra Trees. The optimal hyperparameters were identified, where n_estimators set to 300, max_depth and max_features set to None. This configuration achieved an impressive R² score of 0.994. The training process was conducted separately for both the position and orientation models to determine whether different parameter sets would be more suitable for each. Once more, both models converged on the same optimal parameters. The evaluation results are summarized in Table 4.3.

Table 4.3 *Evaluation Metric of Extra Trees Regressor Model*

Position	
R2 Score	0.994
Mean Squared Error	1.997
Error Position	0.923 mm
Orientation	
R2 Score	0.972
Mean Squared Error	0.000344
Error Orientation	9.116°

5. Discussion

As shown in Table 5.1, Extra Trees Regressor achieved the highest R^2 score, which is equal to 0.994 and the lowest positional error 0.923 mm, indicating excellent accuracy and generalization. Random Forest Regressor also performed very well, but slightly below Extra Trees. Linear Regression, while decent, lagged significantly behind the ensemble models, suggesting that the relationship between input features and tip position is non-linear and better captured by tree-based methods.

Table 5.1 Position Prediction Model Comparison

Model	R2 Score	Error Position (mm)
Linear Regression	0.9199	3.451
Random Forest Regressor	0.991	1.17
Extra Trees Regressor	0.994	0.923 mm

All models demonstrated better performance in predicting the position of CTCR tip compared to its orientation, as shown on Figure 5.2, which can be attributed to the inherently more complex nature of rotational kinematics. Position prediction involves straightforward linear relationships in 3D space, making it easier for regression models to learn. In contrast, orientation prediction deals with non-linear, cyclic data such as angles or quaternions, which are more sensitive to small errors and harder to model accurately. Additionally, orientation representations often introduce challenges like discontinuities and ambiguity (e.g., angle wrapping), which can confuse models if not properly handled. As a result, even advanced ensemble methods like Random Forest and Extra Trees tend to achieve lower accuracy in orientation tasks, highlighting the need for more specialized techniques when modeling rotational behavior in continuum robots.

Table 5.2 Orientation Prediction Model Comparison

Model	R2 Score	Error Orientation (°)
Linear Regression	0.832	14.17
Random Forest Regressor	0.965	8.968
Extra Trees Regressor	0.972	9.116

6. Summary

Among the models evaluated, Extra Trees Regressor demonstrated the best performance in predicting both the position and orientation of the Concentric Tube Continuum Robot (CTCR), outperforming both Linear Regression and Random Forest Regressor. Overall, all models showed better accuracy in predicting position compared to orientation. While the results did not surpass those reported in the benchmark paper, Extra Trees proved to be a strong candidate for fast and effective data-driven forward kinematics modeling of CTCRs.

References

- [1] R. M. Grassmann, R. Z. Chen, N. Liang, and J. Burgner-Kahrs, “A Dataset and Benchmark for Learning the Kinematics of Concentric Tube Continuum Robots,” *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9550–9557, Oct. 2022, doi: <https://doi.org/10.1109/iro547612.2022.9981719>.
- [2] A. Cutler, D. R. Cutler, and J. R. Stevens, “Random Forests,” *Ensemble Machine Learning*, vol. 45, no. 1, pp. 157–175, 2012, doi: https://doi.org/10.1007/978-1-4419-9326-7_5.
- [3] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, no. 1, pp. 3–42, Mar. 2006, doi: <https://doi.org/10.1007/s10994-006-6226-1>.